

P5 — src/utils y configuración

En una frase: el módulo `src/utils` junta las tres piezas de infraestructura que todo script del proyecto usa antes de arrancar —leer la configuración (`load_config`), fijar la semilla de aleatoriedad (`set_seed`) y unificar el estilo de las figuras (`set_plot_style`)— mientras que `configs/default.yaml` es el **panel de control** único donde viven todos los valores del experimento.

Para qué existe este módulo

En un proyecto de identificación de parámetros como este, hay dos cosas que no querés hardcodear desperdigadas por el código: **los números del experimento** (parámetros de WC, estímulos, hiperparámetros de entrenamiento) y **el ritual de arranque** (semilla y estilo de plots). Si eso está disperso, cada script termina con su propia copia de los valores y los resultados dejan de ser reproducibles.

`src/utils` resuelve exactamente eso:

- **config.py** -> una función para leer el YAML de configuración a un diccionario de Python.
- **seed.py** -> fijar semillas (reproducibilidad) y estandarizar el look de las figuras.
- **configs/default.yaml** -> el archivo que esas funciones leen; el único lugar donde se tocan los números.

El patrón de uso desde los scripts (ver **P6**) es siempre el mismo: `cfg = load_config(...)`, después `set_seed(cfg["seed"])`, opcionalmente `set_plot_style()`, y a partir de ahí todo lo demás lee del `cfg`.

Nota sobre el estado del código. Al momento de documentar, las tres funciones (`load_config`, `set_seed`, `set_plot_style`) tienen la firma y el docstring definidos pero el cuerpo es `raise NotImplementedError` —son *stubs* pendientes de implementar. Este manual describe **el contrato** de cada una (qué firma tienen, qué reciben, qué deben devolver), que es lo que fija cómo se las llama desde el resto del proyecto.

Qué expone el paquete (`__init__.py`)

`src/utils/__init__.py` reexporta lo de uso más frecuente para que se pueda importar directo desde el paquete:

```
from .config import load_config # noqa: F401
from .seed import set_seed      # noqa: F401
```

O sea que desde un script podés hacer `from src.utils import load_config, set_seed` sin tener que apuntar al submódulo. Notá que **`set_plot_style` no está reexportada** en el `__init__`: si la necesitás, se importa explícitamente con `from src.utils.seed import set_plot_style`.

Manual función por función

`config.py`

`load_config`

```
def load_config(path: str | Path) -> dict[str, Any]:
    """Lee un archivo YAML de configuración y lo devuelve como dict."""
```

- **Qué hace:** abre el archivo YAML apuntado por `path`, lo parsea y devuelve su contenido como un diccionario de Python.
- **Entrada:** `path` — la ruta al archivo de configuración, como `str` o como `pathlib.Path` (típicamente `configs/default.yaml`).
- **Salida:** un `dict[str, Any]` con toda la config. Las secciones anidadas del YAML (`model`, `stimulus`, `data`, `pinn`, `train`) quedan como sub-diccionarios, así que se accede con `cfg["model"]["wEE"]`, `cfg["train"]["lr"]`, etc.

- **Por qué importa:** es la puerta de entrada de todo. Ningún script debería tener números pegados en el código; en cambio pide `cfg = load_config("configs/default.yaml")` y de ahí saca todo. Cambiar un experimento = editar el YAML, no el código.

seed.py

set_seed

```
def set_seed(seed: int) -> None:
    """Fija las semillas de random, numpy y torch."""
```

- **Qué hace:** fija la semilla de los tres generadores de números aleatorios que usa el proyecto —el `random` de la librería estándar, **NumPy** y **PyTorch (torch)**— para que las corridas sean reproducibles.
- **Entrada:** `seed` — un entero. En la práctica se le pasa `cfg["seed"]` (por defecto 42, ver YAML).
- **Salida:** `None` (efecto de lado: deja los RNG en un estado conocido).
- **Por qué importa:** en este proyecto se generan **datos sintéticos con ruido** y se entrenan redes (Neural ODE / PINN) cuya inicialización es aleatoria. Sin semilla fija, dos corridas del mismo experimento darían trayectorias de ruido y pesos iniciales distintos, y no podrías comparar resultados (por ejemplo, el error por parámetro vs. nivel de ruido) de forma limpia. `set_seed` se llama al principio, justo después de cargar la config.

set_plot_style

```
def set_plot_style() -> None:
    """Estilo común para las figuras (matplotlib)."""
```

- **Qué hace:** aplica un estilo unificado de **matplotlib** (tamaños de fuente, colores, grillas, etc.) para que todas las figuras del proyecto salgan con la misma pinta.
- **Entrada:** ninguna.
- **Salida:** `None` (efecto de lado: modifica la configuración global de `matplotlib`).
- **Por qué importa:** el paquete de estudio y el repo producen muchas figuras (series temporales, retratos de fase, elipses SVD, resultados de control en lazo cerrado...). Centralizar el estilo evita que cada script defina el suyo y garantiza coherencia visual. Se llama una sola vez, antes de empezar a graficar. Ojo: **no** está reexportada en `__init__.py`, se importa desde `src.utils.seed`.

configs/default.yaml — el panel de control

Este es el archivo que `load_config` lee. Es la fuente única de verdad de todos los números del experimento. Está agrupado por secciones; abajo se explica **cada campo**, qué controla y su valor por defecto.

Nivel raíz

Campo	Valor por defecto	Qué controla
seed	42	Semilla global de aleatoriedad. Es lo que se le pasa a <code>set_seed</code> . Fijarla hace reproducibles el ruido de los datos y la inicialización de las redes.

model — los 10 parámetros de Wilson-Cowan

Son exactamente los 10 parámetros que el proyecto busca identificar. Los valores por defecto vienen del simulador de MATLAB y, con los estímulos adecuados, **generan oscilaciones sostenidas (ciclo límite)**. El estado del modelo es `[I, E]` (poblaciones inhibitoria y excitatoria). Ver **B3** (modelo de Wilson-Cowan) y **M2** para el detalle dinámico.

Campo	Valor por defecto	Qué controla
te	1.0	Constante de tiempo de la población excitatoria (τ_e). Qué tan rápido responde E.
ti	2.0	Constante de tiempo de la población inhibitoria (τ_i). E es más rápida que I.
wEE	6.4	Peso sináptico E->E (autoexcitación).
wEI	4.8	Peso sináptico I->E (cuánto frena I a E).
wIE	6.0	Peso sináptico E->I (cuánto activa E a I).
wII	1.2	Peso sináptico I->I (autoinhibición). Es el parámetro difícil de identificar , el cuello de botella del proyecto.
ae	1.2	Ganancia de la sigmoidea excitatoria (a_e). Qué tan abrupta es la curva S de E.
ai	1.0	Ganancia de la sigmoidea inhibitoria (a_i).
thetae	2.8	Umbral de la sigmoidea excitatoria (θ_e). Dónde está el “medio punto” de la curva de E.
thetai	4.0	Umbral de la sigmoidea inhibitoria (θ_i).

stimulus — estímulos externos

Estímulos externos modelados como **pulsos cuadrados** (como en el simulador de MATLAB). P va a la población E; Q va a la población I. Cada uno se define con tres subcampos: amplitud, tiempo de encendido (t_on) y de apagado (t_off). El pulso vale su amplitud entre t_on y t_off, y cero afuera —forma pensada para ser **realizable con optogenética** (on/off, no negativa).

Campo	Valor por defecto	Qué controla
P.amplitude	0.8	Amplitud del pulso hacia E.
P.t_on	100.0	Instante en que se enciende P.
P.t_off	400.0	Instante en que se apaga P.
Q.amplitude	0.6	Amplitud del pulso hacia I.
Q.t_on	200.0	Instante en que se enciende Q.
Q.t_off	500.0	Instante en que se apaga Q.

Ojo con el régimen. Con $P=0.8$, $Q=0.6$ (los valores de este YAML) el sistema cae en un **foco estable**; para forzar oscilación sostenida (ciclo límite) hacen falta valores tipo $P \approx 1.2$, $Q \approx 0.6$. No confundir la config por defecto con el caso oscilante.

data — generación de datos

Controla cómo se integra el modelo para producir el dataset sintético.

Campo	Valor por defecto	Qué controla
t_span	[0.0, 600.0]	Ventana temporal de la simulación (inicio y fin).
I0	0.0	Condición inicial de la población inhibitoria.
E0	0.0	Condición inicial de la población excitatoria. Arranca en reposo E=I=0.
rel_tol	1.0e-3	Tolerancia relativa del integrador de EDOs.
abs_tol	1.0e-6	Tolerancia absoluta del integrador.
n_eval	6000	Cantidad de puntos de la grilla uniforme de muestreo. null -> usar los pasos adaptativos del integrador en vez de una grilla fija.
noise_std	0.0	Desvío estándar del ruido gaussiano agregado a las observaciones. 0.0 = datos limpios; se sube para los experimentos de robustez al ruido.
out_path	data/processed/dataset.npz	Ruta donde se guarda el dataset generado.

pinn — arquitectura de la red

Define la MLP que usa la variante **PINN** (ver **P4** / **P6**).

Campo	Valor por defecto	Qué controla
in_dim	1	Dimensión de entrada de la red (el tiempo t).
out_dim	2	Dimensión de salida (las dos poblaciones [I, E]).
hidden_dim	64	Neuronas por capa oculta.
n_layers	4	Cantidad de capas.

train — hiperparámetros de entrenamiento

Controla el bucle de optimización.

Campo	Valor por defecto	Qué controla
epochs	10000	Número de épocas de entrenamiento.
lr	0.001	Learning rate (tasa de aprendizaje del optimizador).
w_data	1.0	Peso del término de pérdida por ajuste a los datos .
w_physics	1.0	Peso del término de pérdida por residuo físico (que se cumpla la EDO de WC).
w_ic	1.0	Peso del término de pérdida por condición inicial .
n_collocation	1000	Cantidad de puntos de colocación donde se evalúa el residuo físico (PINN).

Campo	Valor por defecto	Qué controla
device	cpu	Dispositivo de cómputo (cpu o cuda).
log_every	100	Cada cuántas épocas se loguea el progreso.
checkpoint_dir	results/models	Carpeta donde se guardan los checkpoints del modelo.

Cómo se usan estas piezas desde los scripts (P6)

El flujo típico al arrancar cualquier script del proyecto es:

```
from src.utils import load_config, set_seed
from src.utils.seed import set_plot_style # no está en __init__, se importa aparte

cfg = load_config("configs/default.yaml") # todo el panel de control en un dict
set_seed(cfg["seed"])                    # reproducibilidad (semilla 42 por defecto)
set_plot_style()                          # figuras con estilo unificado

# De acá en más todo lee del cfg:
tau_e = cfg["model"]["te"]
p_amp = cfg["stimulus"]["P"]["amplitude"]
epochs = cfg["train"]["epochs"]
out = cfg["data"]["out_path"]
```

La idea de fondo: **el código no tiene números, el YAML sí**. Para correr una variante del experimento (más ruido, otra semilla, otra arquitectura de red, otro learning rate) editás `configs/default.yaml` y volvés a correr —sin tocar la lógica. Eso es lo que mantiene los resultados comparables y reproducibles a lo largo de todo el paquete.

Ver **P6** para los scripts concretos que consumen esta configuración.